```
var macaroon = MacaroonsBuilder.deserialize(tokenId); #D
var verifier = new MacaroonsVerifier(macaroon); #D
if (verifier.isValid(macKey.getEncoded())) { #D
return delegate.read(request, macaroon.identifier); #E
}
return Optional.empty();
}
@Override
public void revoke(Request request, String tokenId) {
var macaroon = MacaroonsBuilder.deserialize(tokenId);
delegate.revoke(request, macaroon.identifier); #F
}
}
```

#A Use another token store to create a unique identifier for this macaroon.
#B Create the macaroon with a location hint, the identifier, and the authentication key.
#C Return the serialized URL-safe string form of the macaroon.
#D Deserialize and validate the macaroon signature and caveats.
#E If the macaroon is valid then lookup the identifier in the delegate token store.
#F To revoke a macaroon, revoke the identifier in the delegate store.

You can now wire up the `CapabilityController` to use the new token store for capability tokens. Open the Main.java file in your editor and find the lines that construct the `CapabilityController`. Update the file to use the `MacaroonTokenStore` instead. You may need to first move the code that reads the `macKey` from the keystore (see chapter 6) from later in the file. The code should look as follows, with the new part highlighted in bold: